PYTHON PROGRAMMING

ACTIVITY - BASED LEARNING

10 Case Studies with Solutions

• Name: Damu Hari Kaushik

• Roll No: 26000885

• Branch: Computer Science &

Engineering (AI & ML)

• Section: CSM1

• Course: Python Programming

• Institution: Dr . RVR NRI Institute of

Technology

Contents : -

1. Case Study 1: University Scholarship Allocation

2. Case Study 2: Progressive Electricity Billing

3. Case Study 3: Hospital Billing and Patient Classification

4. Case Study 4: Employee Payroll and Deduction Analysis

5. Case Study 5: Multi -Plan Telecommunications Cost Optimization

6. Case Study 6: Railway Fare and Reservation Calculator

7. Case Study 7: Retail Billing with Multiple Discount Rules

8. Case Study 8 : Manufacturing Cost and Profitability Analysis

9. Case Study 9: Multi -Currency Travel Budget Analysis

10. Case Study 10: Campus Event Financial Planning

CASE STUDY 1: University Scholarship Allocation

1. Problem Statement

Develop a program to evaluate scholarship applicants using aggregate marks, subject-wise minimum marks, attendance, and family income. Rank eligible students and display the result.

2. Objectives

• Read applicant details and marks.

• Calculate aggregate percentage.

• Check subject minimum, attendance, and income conditions.

• Classify applicants as eligible or not eligible and show the result.

3. Assumptions

• Three subject marks are entered, each from 0 to 100.

• Every subject must have at least 50 marks.

• Attendance must be at least 75%.

• Family income must not exceed 300,000.

• Eligible applicants are evaluated based on the criteria thresholds.

4. Algorithm

1. Start.

2. Input applicant name, three marks, attendance, and family income.

3. Check whether marks are between 0 and 100 and other values are valid.

4. Calculate total and percentage.

5. Check all scholarship conditions using conditional statements.

6. Display eligibility and aggregate percentage.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS

|

▼

VALIDATE INPUTS

|

▼

CALCULATE USING CONDITIONS / LOOPS

|

▼

DISPLAY RESULT

|

▼

STOP

6. Pseudocode

Plaintext

START

INPUT name, m1, m2, m3, attendance, income

total = m1 + m2 + m3

percentage = total / 3

IF any mark < 0 OR mark > 100 OR attendance < 0 OR attendance > 100 OR income < 0

THEN

PRINT "Invalid input. "

ELSE

IF m1 < 50 OR m2 < 50 OR m3 < 50 OR attendance < 75 OR income > 300000 THEN

PRINT "Result: Not eligible for scholarship"

ELSE

PRINT "Result: Eligible for scholarship"

END IF

END IF

STOP

7. Python Source Code (case_study_3.py)

```
name = input("Applicant name: ")

m1 = float(input("Subject 1 marks: "))

m2 = float(input("Subject 2 marks: "))

m3 = float(input("Subject 3 marks: "))

attendance = float(input("Attendance percentage: "))

income = float(input("Family annual income (Rs.): "))

if (m1 < 0 or m1 > 100 or m2 < 0 or m2 > 100 or m3 < 0 or m3 > 100 or attendance < 0 or

attendance > 100 or income < 0):

print("Invalid input. ")

else:

percentage = (m1 + m2 + m3) / 3

print("Applicant:" , name)

print("Aggregate percentage:" , round(percentage, 2))

if m1 >= 50 and m2 >= 50 and m3 >= 50 and attendance >= 75 and income <= 300000:

print("Result: Eligible for scholarship")

else:

print("Result: Not eligible for scholarship")
```

8. Test Cases

Input / Situation Expected Result Purpose

Anu, 80, 75, 90, 85,

200000 81.67%; Eligible Normal case

Input / Situation Expected Result Purpose

Ravi, 45, 80, 70, 90,

150000 Not eligible (subject below 50) Minimum-mark

check

Maya, 70, 70, 70, 60,

100000

Not eligible (attendance below

75%) Attendance check

9. Output screenshots

10. Conclusion

The program successfully uses variables, arithmetic operators, and conditional

statements to check scholarship eligibility according to predefined criteria.

CASE STUDY 2: Progressive Electricity Billing

1. Problem Statement

Calculate a household electricity bill using multiple consumption slabs, a fixed charge,

and a surcharge for high consumption.

2. Objectives

• Read units consumed.

• Calculate charges slab by slab.

• Add the fixed charge and any surcharge.

• Print a bill summary.

3. Assumptions

• First 100 units cost 2 per unit.

• Next 100 units cost 3 per unit.

• Units above 200 cost 5 per unit.

• Fixed charge is 50.

• Consumption above 200 units has a 5% surcharge on energy charge.

4. Algorithm

1. Start.

2. Input units consumed.

3. Reject negative units.

4. Calculate energy charge using the correct slab.

5. Calculate fixed charge and surcharge.

6. Add all charges and display the bill.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS

|

▼

VALIDATE INPUTS

|

▼

CALCULATE USING CONDITIONS / LOOPS

|

▼

DISPLAY RESULT

|

▼

STOP

6. Pseudocode

Plaintext

START

INPUT units

IF units < 0:

PRINT error

ELSE IF units <= 100:

energy = units * 2

ELSE IF units <= 200:

energy = 100 * 2 + (units - 100) * 3

ELSE:

energy = 100 * 2 + 100 * 3 + (units - 200) * 5

fixed = 50

IF units > 200:

```
surcharge = energy * 0.05

ELSE:

surcharge = 0

total = energy + fixed + surcharge

PRINT energy, fixed, surcharge, total

STOP
```

7. Python Source Code (case_study_2.py)

```
units = int(input("Enter electricity units: "))

if units < 0:

print("Invalid units. ")

else:

if units <= 100:

energy = units * 2

elif units <= 200:

energy = 100 * 2 + (units - 100) * 3

else:

energy = 100 * 2 + 100 * 3 + (units - 200) * 5

if units > 200:

fixed = 50

surcharge = energy * 0.05

else:

fixed = 50

surcharge = 0

total = energy + fixed + surcharge
```

```
print("\n--- Electricity Bill ---")

print("Energy charge: Rs. " , round(energy, 2))

print("Fixed charge: Rs. " , fixed)

print("Surcharge: Rs. " , round(surcharge, 2))

print("Total bill: Rs. " , round(total, 2))
```

8. Test Cases

Input / Situation Expected Result Purpose

80 Energy 160; total 210 Normal

150 Energy 350; total 400 Middle slab

250 Energy 750; surcharge 37.50; total 837.50 High slab

9. Output Screenshots

10. Conclusion

The program demonstrates progressive slab calculations, conditions, and arithmetic operators.

CASE STUDY 3: Hospital Billing and Patient Classification

1. Problem Statement

Calculate consultation, diagnostic, and treatment charges, apply a concession when eligible, validate inputs, and classify the bill using the final amount.

2. Objectives

• Read the three charge amounts.

• Validate that charges are non-negative.

• Apply a concession for senior patients.

• Calculate the final amount and classify the bill.

3. Assumptions

• Consultation, diagnostics, and treatment charges cannot be negative.

• Senior patients aged 60 or above receive a 10% concession on the subtotal.

• Bills below 1000 are Basic, 1000 to 4999 are Standard, and 5000 or more are High.

4. Algorithm

1. Start.

2. Input age and the three charges.

3. Validate all inputs.

4. Add charges to find subtotal.

5. Apply 10% concession if age is at least 60.

6. Calculate final bill and classification.

7. Display the bill.

8. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS



VALIDATE INPUTS

CALCULATE USING CONDITIONS / LOOPS

DISPLAY RESULT

STOP

6. Pseudocode

Plaintext

```
START
INPUT age, consultation, diagnostics, treatment
IF age < 0 OR consultation < 0 OR diagnostics < 0 OR treatment < 0 THEN
PRINT "Invalid input. "
ELSE
subtotal = consultation + diagnostics + treatment
IF age >= 60 THEN
discount = subtotal * 0.10
ELSE
discount = 0
END IF
final_bill = subtotal - discount
IF final_bill < 1000 THEN
category = "Basic"
```

```
ELSE IF final_bill < 5000 THEN

category = "Standard"

ELSE

category = "High"

END IF

PRINT subtotal, discount, final_bill, category

END IF

STOP
```

7. Python Source Code (case_study_3.py)

```
age = int(input("Patient age: "))

consult = float(input("Consultation charge: Rs. "))

diagnostic = float(input("Diagnostic charge: Rs. "))

treatment = float(input("Treatment charge: Rs. "))

if age < 0 or consult < 0 or diagnostic < 0 or treatment < 0:

print("Invalid input. ")

else:

subtotal = consult + diagnostic + treatment

if age >= 60:

discount = subtotal * 0.10

else:

discount = 0

final_bill = subtotal - discount

if final_bill < 1000:

category = "Basic"

elif final_bill < 5000:

category = "Standard"

else:

category = "High"
```

```
print("\n--- Hospital Bill ---")

print("Subtotal: Rs. " , round(subtotal, 2))

print("Concession: Rs. " , round(discount, 2))

print("Final bill: Rs. " , round(final_bill, 2))

print("Bill category:" , category)
```

8. Test Cases

Input / Situation Expected Result Purpose

Age 65; 500, 800,

2000

Subtotal 3300; concession 330; final 2970;

Standard

Senior

concession

Age 30; 200, 200,

300 Final 700; Basic No concession

Age 50; -5, 100, 100 Invalid input Validation

9. Output Screenshots

10. Conclusion

The program uses input validation, conditional concessions, and classification of a final

bill.

CASE STUDY 4: Employee Payroll and Deduction Analysis

1. Problem Statement

Calculate gross salary from basic pay, allowances, overtime, and incentives. Apply a deduction and display a formatted salary statement.

2. Objectives

• Read basic pay, overtime hours, and incentive.

• Calculate allowances and overtime pay.

• Calculate gross salary, deduction, and net salary.

• Display the salary statement.

3. Assumptions

• HRA is 20% of basic pay.

• Other allowance is 10% of basic pay.

• Overtime is 150 per hour.

• Incentive is added to gross salary.

• Deduction is 8% of gross salary.

4. Algorithm

1. Start.

2. Input basic pay, overtime hours, and incentive.

3. Reject negative values.

4. Calculate HRA, other allowance, and overtime pay.

5. Calculate gross salary and deduction.

6. Calculate net salary and display details.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS

|

▼

VALIDATE INPUTS

|

▼

CALCULATE USING CONDITIONS / LOOPS

|

▼

DISPLAY RESULT

|

▼

STOP

6. Pseudocode

Plaintext

START

INPUT basic, overtime_hours, incentive

IF basic < 0 OR overtime_hours < 0 OR incentive < 0 THEN

PRINT error

ELSE

hra = basic * 0.20

other = basic * 0.10

overtime = overtime_hours * 150

gross = basic + hra + other + overtime + incentive

deduction = gross * 0.08

net = gross - deduction

PRINT salary details

END IF

STOP

7. Python Source Code (case_study_4.py)

Python

```
basic = float(input("Basic pay (Rs.): "))

hours = float(input("Overtime hours: "))

incentive = float(input("Incentive (Rs.): "))

if basic < 0 or hours < 0 or incentive < 0:

print("Invalid input. ")

else:

hra = basic * 0.20

other = basic * 0.10

overtime = hours * 150

gross = basic + hra + other + overtime + incentive

deduction = gross * 0.08

net = gross - deduction

print("\n--- Salary Statement ---")

print("Basic pay: Rs. " , round(basic, 2))

print("HRA: Rs. " , round(hra, 2))

print("Other allowance: Rs. " , round(other, 2))

print("Overtime pay: Rs. " , round(overtime, 2))

print("Incentive: Rs. " , round(incentive, 2))

print("Gross salary: Rs. " , round(gross, 2))

print("Deduction: Rs. " , round(deduction, 2))

print("Net salary: Rs. " , round(net, 2))
```

8. Test Cases

Input / Situation Expected Result Purpose

Basic 20000; 2 hours;

incentive 500

Gross 26800; deduction 2144;

net 24656 Normal

Basic 15000; 0 hours;

incentive 0

Gross 19500; deduction 1560;

net 17940

No

overtime

Basic -100 Invalid input Validation

9. Output Screenshots

10. Conclusion

The program demonstrates salary calculations, percentages, input validation, and

formatted output

CASE STUDY 5: Multi-Plan Telecommunications Cost Optimization

1. Problem Statement

Compare three mobile plans using monthly usage and budget. Calculate excess data charges and identify the least expensive plan within the user's budget.

2. Objectives

• Read monthly data usage and budget.

• Calculate each plan's monthly cost.

• Compare plans against the budget.

• Display the cheapest eligible plan.

3. Assumptions

• Plan A: 199 includes 2 GB; extra data costs 50 per GB.

• Plan B: 299 includes 5 GB; extra data costs 30 per GB.

• Plan C: 399 includes 10 GB; extra data costs 20 per GB.

• Usage and budget cannot be negative.

• The cheapest plan within budget is recommended.

4. Algorithm

1. Start.

2. Input usage in GB and budget.

3. Validate the values.

4. Calculate the cost of each plan, including excess data.

5. Check which costs are within budget.

6. Choose the lowest eligible cost or report no eligible plan.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS



VALIDATE INPUTS



CALCULATE USING CONDITIONS / LOOPS



DISPLAY RESULT



STOP

6. Pseudocode

Plaintext

START

INPUT usage, budget

IF usage < 0 OR budget < 0 THEN

PRINT error

ELSE

cost_a = 199 + max(0, usage - 2) * 50

cost_b = 299 + max(0, usage - 5) * 30

cost_c = 399 + max(0, usage - 10) * 20

Find cheapest cost within budget

PRINT recommended plan and cost

END IF

STOP

7. Python Source Code (case_study_5.py)

Python

```
usage = float(input("Monthly data usage (GB): "))
budget = float(input("Monthly budget (Rs.): "))
if usage < 0 or budget < 0:
print("Invalid input. ")
else:
cost_a = 199 + max(0, usage - 2) * 50
cost_b = 299 + max(0, usage - 5) * 30
cost_c = 399 + max(0, usage - 10) * 20
plans = [("Plan A" , cost_a), ("Plan B" , cost_b), ("Plan C" , cost_c)]
print("Plan A cost: Rs. " , round(cost_a, 2))
print("Plan B cost: Rs. " , round(cost_b, 2))
print("Plan C cost: Rs. " , round(cost_c, 2))
best_name = ""
best_cost = 0
for name, cost in plans:
if cost <= budget and (best_name == "" or cost < best_cost):
best_name = name
best_cost = cost
if best_name == "":
print("No plan is within your budget. ")
else:
```

```
print("Recommended plan:" , best_name)

print("Monthly cost: Rs. " , round(best_cost, 2))
```

8. Test Cases

Input / Situation Expected Result Purpose

Usage 4 GB; budget 350 Plan A or B at 299 Compare plans

Usage 12 GB; budget 600 Plan C 439 High usage

Usage -1 GB Invalid input Validation

9. Output Screenshots

10. Conclusion

The program uses lists, loops, arithmetic, and conditions to compare plan costs under a budget.

CASE STUDY 6: Railway Fare and Reservation Calculator

1. Problem Statement

Calculate railway ticket fare based on distance, class of travel, and passenger category,

applying senior citizen or child concessions.

2. Objectives

• Read distance, travel class, and age.

• Calculate base fare using distance and class rate.

• Apply age-based concessions.

• Display the final fare.

3. Assumptions

• Sleeper class is 1.50 per km, AC Three Tier is 2.50 per km, and AC First Class is

4.00 per km.

• Children under 5 travel free (0 fare).

• Senior citizens (60 and above) receive a 25% concession on the base fare.

• Distance and age cannot be negative.

4. Algorithm

1. Start.

2. Input distance, class type, and age.

3. Validate inputs.

4. Calculate base fare according to the class rate.

5. Apply concessions based on age.

6. Display final ticket fare.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS



VALIDATE INPUTS

CALCULATE USING CONDITIONS / LOOPS

DISPLAY RESULT

STOP

6. Pseudocode

START

INPUT distance, travel_class, age

IF distance < 0 OR age < 0 THEN

PRINT error

ELSE

IF travel_class == 1 THEN

rate = 1.50

ELSE IF travel_class == 2 THEN

rate = 2.50

ELSE

rate = 4.00

END IF

base_fare = distance * rate

IF age < 5 THEN

final_fare = 0

```
ELSE IF age >= 60 THEN

final_fare = base_fare * 0.75

ELSE

final_fare = base_fare

END IF

PRINT final_fare

END IF

STOP
```

7. Python Source Code (case_study_6.py)

```
distance = float(input("Distance (km): "))

print("Classes: 1. Sleeper, 2. AC 3-Tier, 3. AC First Class")

travel_class = int(input("Enter class choice (1-3): "))

age = int(input("Passenger age: "))

if distance < 0 or age < 0 or travel_class < 1 or travel_class > 3:

print("Invalid input. ")

else:

if travel_class == 1:

rate = 1.50

elif travel_class == 2:

rate = 2.50

else:

rate = 4.00

base_fare = distance * rate

if age < 5:

final_fare = 0

elif age >= 60:

final_fare = base_fare * 0.75

else:
```

```
final_fare = base_fare

print("\n--- Railway Ticket Summary ---")

print("Base fare: Rs. " , round(base_fare, 2))

print("Final ticket fare: Rs. " , round(final_fare, 2))
```

8. Test Cases

Input / Situation Expected Result Purpose

Distance 500 km; Sleeper (1); age

30 Base 750; final 750 Normal

Distance 800 km; AC 3-Tier (2); age

65

Base 2000; concession 500; final

1500 Senior citizen

Distance 300 km; AC First (3); age 3 Final fare 0 Child free

travel

9. Output Screenshots

10. Conclusion

The program successfully uses multi-way conditional statements, rate formulas, and

age-based checking.

CASE STUDY 7: Retail Billing with Multiple Discount Rules

1. Problem Statement

Calculate total retail purchase amount, apply a seasonal or promotional discount

based on purchase value and membership status, and compute net payable amount.

2. Objectives

• Read item price and quantity for three items.

• Check membership status.

• Compute subtotal and apply tiered percentage discounts.

• Display final billing statement.

3. Assumptions

• Prices and quantities cannot be negative.

• Purchases of 5000 or more receive an extra 5% discount.

• Premium members receive an additional 10% discount.

4. Algorithm

1. Start.

2. Input prices, quantities, and membership status.

3. Validate inputs.

4. Compute subtotal.

5. Apply discount rules based on amount and membership.

6. Calculate net amount and display bill.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS

|

▼

VALIDATE INPUTS

|

▼

CALCULATE USING CONDITIONS / LOOPS

|

▼

DISPLAY RESULT

|

▼

STOP

6. Pseudocode

Plaintext

START

INPUT p1, q1, p2, q2, p3, q3, is_member

subtotal = (p1 * q1) + (p2 * q2) + (p3 * q3)

discount = 0

IF subtotal >= 5000 THEN

discount = discount + (subtotal * 0.05)

END IF

IF is_member == "yes" THEN

discount = discount + (subtotal * 0.10)

END IF

net_amount = subtotal - discount

PRINT subtotal, discount, net_amount

STOP

7. Python Source Code (case_study_7.py)

Python

```
p1 = float(input("Item 1 price: "))
q1 = int(input("Item 1 quantity: "))
p2 = float(input("Item 2 price: "))
q2 = int(input("Item 2 quantity: "))
p3 = float(input("Item 3 price: "))
q3 = int(input("Item 3 quantity: "))
is_member = input("Premium member? (yes/no): ").lower()
if p1 < 0 or q1 < 0 or p2 < 0 or q2 < 0 or p3 < 0 or q3 < 0:
print("Invalid input. ")
else:
subtotal = (p1 * q1) + (p2 * q2) + (p3 * q3)
discount = 0
if subtotal >= 5000:
discount += subtotal * 0.05
if is_member == "yes":
discount += subtotal * 0.10
net_amount = subtotal - discount
print("\n--- Retail Bill Summary ---")
print("Subtotal: Rs. " , round(subtotal, 2))
print("Total discount: Rs. " , round(discount, 2))
print("Net payable: Rs. " , round(net_amount, 2))
```

8. Test Cases

Input / Situation Expected Result Purpose

Prices 1000, 2000, 500; quantities

1, 1, 2; member: no

Subtotal 4000; discount

0; net 4000

Below

threshold

Prices 3000, 2000, 1000;

quantities 1, 1, 1; member: yes

Subtotal 6000; discount

15%; net 5100

High value +

member

9. Output Screenshots

10. Conclusion

The program effectively applies multiple conditional discount rules and aggregates

shopping cart totals.

CASE STUDY 8: Manufacturing Cost and Profitability Analysis

1. Problem Statement

Calculate total manufacturing cost from material, labor, and overhead costs, add a target profit margin, and compute unit selling price.

2. Objectives

• Read material cost, labor cost, overheads, and units produced.

• Calculate total manufacturing cost and cost per unit.

• Add profit margin to determine selling price.

• Display financial summary.

3. Assumptions

• All cost inputs and units must be positive non-zero values.

• Profit margin is 20% of the total manufacturing cost.

4. Algorithm

1. Start.

2. Input material cost, labor cost, overhead cost, and units produced.

3. Validate inputs.

4. Compute total manufacturing cost and cost per unit.

5. Add 20% profit margin to find total revenue required.

6. Calculate selling price per unit and display summary.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS

|

▼

VALIDATE INPUTS

|

▼

CALCULATE USING CONDITIONS / LOOPS

|

▼

DISPLAY RESULT

|

▼

STOP

6. Pseudocode

Plaintext

START

INPUT material, labor, overhead, units

IF material < 0 OR labor < 0 OR overhead < 0 OR units <= 0 THEN

PRINT error

ELSE

total_cost = material + labor + overhead

cost_per_unit = total_cost / units

profit = total_cost * 0.20

total_revenue = total_cost + profit

selling_price = total_revenue / units

PRINT total_cost, cost_per_unit, selling_price

END IF

STOP

7. Python Source Code (case_study_8.py)

Python

material = float(input("Material cost: Rs. "))

```
labor = float(input("Labor cost: Rs. "))
overhead = float(input("Overhead cost: Rs. "))
units = int(input("Units produced: "))
if material < 0 or labor < 0 or overhead < 0 or units <= 0:
print("Invalid input. ")
else:
total_cost = material + labor + overhead
cost_per_unit = total_cost / units
profit = total_cost * 0.20
total_revenue = total_cost + profit
selling_price = total_revenue / units
print("\n--- Manufacturing Summary ---")
print("Total manufacturing cost: Rs. " , round(total_cost, 2))
print("Cost per unit: Rs. " , round(cost_per_unit, 2))
print("Target profit (20%): Rs. " , round(profit, 2))
print("Selling price per unit: Rs. " , round(selling_price, 2))
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Material 50000; labor 30000; overhead 20000; 1000 units	Total cost 100000; unit cost 100; selling price 120	Normal production
Units 0	Invalid input (division by zero prevention)	Zero unit check

9. Output Screenshots

10. Conclusion

The program calculates manufacturing overheads, profit margins, and per-unit pricing with input validation.

CASE STUDY 9: Multi-Currency Travel Budget Analysis

1. Problem Statement

Convert a traveler's budget in Indian Rupees into three foreign currencies and check if estimated trip expenses fit within the converted budget.

2. Objectives

• Read budget in INR and estimated expenses in foreign currency.

• Convert INR to USD, EUR, and GBP using fixed exchange rates.

• Compare estimated expenses with converted budget.

• Display budget status.

3. Assumptions

• 1 USD = 83 INR, 1 EUR = 90 INR, 1 GBP = 105 INR.

• Budget and expenses cannot be negative.

4. Algorithm

1. Start.

2. Input budget in INR and choice of foreign currency.

3. Validate inputs.

4. Perform conversion based on exchange rates.

5. Compare with estimated trip expenses.

6. Display status and result.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS

|

▼

VALIDATE INPUTS

CALCULATE USING CONDITIONS / LOOPS



DISPLAY RESULT

STOP

6. Pseudocode

Plaintext

```
START
INPUT inr_budget, currency_choice, expense
IF inr_budget < 0 OR expense < 0 THEN
PRINT error
ELSE
IF currency_choice == 1 THEN
converted = inr_budget / 83
currency_name = "USD"
ELSE IF currency_choice == 2 THEN
converted = inr_budget / 90
currency_name = "EUR"
ELSE
converted = inr_budget / 105
currency_name = "GBP"
END IF
IF expense <= converted THEN
```

```
PRINT "Within budget"

ELSE

PRINT "Exceeds budget"

END IF

END IF

STOP
```

7. Python Source Code (case_study_9.py)

Python

```
inr_budget = float(input("Enter budget in INR: "))

print("Choose currency: 1. USD, 2. EUR, 3. GBP")

choice = int(input("Enter choice (1-3): "))

expense = float(input("Estimated trip expense in foreign currency: "))

if inr_budget < 0 or expense < 0 or choice < 1 or choice > 3:

print("Invalid input. ")

else:

if choice == 1:

converted = inr_budget / 83

c_name = "USD"

elif choice == 2:

converted = inr_budget / 90

c_name = "EUR"

else:

converted = inr_budget / 105

c_name = "GBP"

print("\n--- Travel Budget Summary ---")

print("Converted Budget:" , round(converted, 2), c_name)
```

```
if expense <= converted:
print("Status: Within budget")
else:
print("Status: Budget exceeded")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
Budget 83000 INR; USD (1); expense 900	Converted 1000 USD; Within budget	Normal conversion
Budget 45000 INR; EUR (2); expense 600	Converted 500 EUR; Budget exceeded	Exceeding budget

9. Output Screenshots

10. Conclusion

The program demonstrates currency conversion calculations, selection logic, and budget comparison.

CASE STUDY 10: Campus Event Financial Planning

1. Problem Statement

Calculate total event revenue from ticket sales and sponsorships, subtract fixed and variable expenses, and compute net profit or loss for a campus fest.

2. Objectives

• Read ticket count, ticket price, sponsorship amount, and expenses.

• Calculate total revenue and total expenses.

• Compute net profit or loss.

• Display financial outcome.

3. Assumptions

• Ticket price, counts, sponsorships, and expenses cannot be negative.

• Total Revenue = (Tickets Sold * Ticket Price) + Sponsorships.

• Net Financial Result = Total Revenue - Total Expenses.

4. Algorithm

1. Start.

2. Input tickets sold, ticket price, sponsorships, and total expenses.

3. Validate inputs.

4. Calculate total revenue.

5. Compute net profit or loss.

6. Display event financial statement.

7. Stop.

5. Flowchart

Plaintext

START

|

▼

INPUT DETAILS

|

▼

VALIDATE INPUTS

|

▼

CALCULATE USING CONDITIONS / LOOPS

|

▼

DISPLAY RESULT

|

▼

STOP

6. Pseudocode

Plaintext

```
START
INPUT tickets_sold, price, sponsorship, expenses
IF tickets_sold < 0 OR price < 0 OR sponsorship < 0 OR expenses < 0 THEN
PRINT error
ELSE
revenue = (tickets_sold * price) + sponsorship
net = revenue - expenses
IF net >= 0 THEN
PRINT "Profit:" , net
ELSE
PRINT "Loss:" , abs(net)
END IF
END IF
STOP
```

7. Python Source Code (case_study_10.py)

Python

```
tickets = int(input("Tickets sold: "))
price = float(input("Price per ticket: Rs. "))
sponsorship = float(input("Total sponsorship amount: Rs. "))
expenses = float(input("Total event expenses: Rs. "))
if tickets < 0 or price < 0 or sponsorship < 0 or expenses < 0:
print("Invalid input. ")
else:
revenue = (tickets * price) + sponsorship
net = revenue - expenses
print("\n--- Event Financial Statement ---")
print("Total Revenue: Rs. " , round(revenue, 2))
print("Total Expenses: Rs. " , round(expenses, 2))
if net > 0:
print("Net Profit: Rs. " , round(net, 2))
elif net < 0:
print("Net Loss: Rs. " , round(abs(net), 2))
else:
print("Break-even (No profit, no loss)")
```

8. Test Cases

Input / Situation	Expected Result	Purpose
200 tickets; 100 price; 15000 sponsor; 25000 expenses	Revenue 35000; Profit 10000	Profitable event

Input / Situation Expected Result Purpose

50 tickets; 100 price; 5000 sponsor;

15000 expenses

Revenue 10000; Loss

5000

Loss

scenario

9. Output Screenshots

10. Conclusion

The program successfully computes event revenue, expenses, and net profit or loss

balances.